

PROYECTO DE INGENIERÍA DE CONTROL: Automatización del Nivel en un Tanque de Separación Agua-Crudo (Lazo Cerrado)

Contexto Industrial (Oil & Gas - Production Systems)

En las facilidades de producción en superficie, los tanques separadores de fases (gas-crudo-agua) son activos críticos para garantizar la calidad del hidrocarburo antes de su transporte y asegurar que el agua de producción cumpla con las normativas ambientales antes de su reinyección. Mantener el nivel de la interfaz agua-crudo bajo un control estricto es fundamental: un nivel muy alto provoca arrastre de agua en el crudo (*water carryover*), dañando las refinerías; un nivel muy bajo permite que el crudo se escape por la línea de drenaje de agua (*oil carryunder*).

Este proyecto simula el reto real de diseñar y validar un sistema de control de nivel robusto frente a perturbaciones de proceso para un tanque separador vertical.

Fases del Proyecto:

Parte 1: Modelado Dinámico y Linealización Computacional

El comportamiento dinámico del nivel de agua en el separador se rige por el balance de masa global:

$$A \frac{dh(t)}{dt} = Q_i(t) - C_v \sqrt{h(t)}$$

Donde A es el área transversal del tanque, $h(t)$ es el nivel de la interfaz, Q_i es el caudal de entrada de líquido y el término $C_v = \sqrt{h(t)}$ representa el caudal de salida no lineal a través de la válvula de descarga hidráulica.

1. **Linealización por Software:** Utilizando el **Symbolic Math Toolbox** de MATLAB (comandos `syms`, `jacobian`) o las herramientas de linealización de Simulink (`linearize` o el *Model Linearizer App*), obtenga el modelo linealizado en variables de desviación alrededor del punto de operación nominal $h_0 = 2m$.
2. **Función de Transferencia:** Genere programáticamente la función de transferencia de la planta en lazo abierto $G(s) = \frac{H(s)}{Q_i(s)}$ utilizando el comando `tf()`.

Sintonización Automatizada del Controlador PID

Para cumplir con los criterios de seguridad operativa de la estación de flujo, el lazo cerrado debe cumplir estrictamente con las siguientes especificaciones ante un cambio tipo escalón en la referencia:

- **Nivel de operación nominal:** 2 m con una banda de tolerancia de ± 5 cm.
- **Tiempo de asentamiento (t_s):** ≤ 30 s (criterio del 2%).
- **Sobreimpulso máximo (%OS):** 0% (respuesta sobreamortiguada o críticamente amortiguada para evitar perturbaciones hidrodinámicas que emulsionen el crudo).

Utilizando las herramientas de sintonización automática de MATLAB (como el comando *pidtune*, el *PID Tuner App* o el *Control System Designer*), determine las ganancias óptimas del controlador. El algoritmo de diseño debe priorizar la eliminación del sobreimpulso y garantizar el cumplimiento del tiempo de asentamiento.

Parte 3: Co-Simulación en Simulink con Fenómenos de Planta Reales

Construya un modelo dinámico en Simulink para evaluar el desempeño del controlador sintonizado frente a condiciones adversas de campo. El diagrama de bloques debe incluir:

1. **Bloque de la Planta:** Implementación de la función de transferencia linealizada $G(s)$ u operaciones nativas de Simulink.
2. **Controlador PID Integrado:** Configurado con las ganancias calculadas por software. Debe activar obligatoriamente la opción de **Anti-windup** integrada en el bloque PID (método *clamping* o *back-calculation*) para mitigar los efectos de la saturación de la válvula física.
3. **Señal de Referencia y Perturbación:** Un bloque *Step* para el setpoint hacia los 2 m en el segundo $t = 10s$.
 - Un bloque *Step* aditivo a la salida de la planta en el segundo $t = 1200s$ para simular una perturbación abrupta en el caudal de salida Q_o (por ejemplo, por la apertura repentina de una línea secundaria de drenaje).
4. **Exportación de Datos:** Bloques **To Workspace** configurados en formato *Timeseries* para enviar las señales de tiempo, nivel y esfuerzo de control directo a MATLAB.

- **Tiempo total de simulación:** 2000 s.

Parte 4: Análisis de Estabilidad y Robustez en Frecuencia

A partir del controlador final obtenido por software, analice la robustez del lazo corregido utilizando herramientas de análisis lineal:

1. **Lugar Geométrico de las Raíces (LGR):** Genere el gráfico mediante comandos de MATLAB (*rlocus*) para analizar la estabilidad del lazo compensado.
2. **Análisis en Frecuencia:** Utilice el comando *margin()* para calcular de manera automática el Margen de Ganancia (GM), el Margen de Fase (PM) y sus respectivas frecuencias de cruce (ω_{GM} y ω_{PM}) en lazo abierto.

Parte 5: Postprocesamiento y Automatización mediante Script de MATLAB

Desarrolle un único script maestro de MATLAB (.m) estructurado que automatice la ejecución y la presentación de resultados:

- **Simulación Programática:** El script debe precargar las ganancias, disparar la simulación de Simulink usando el comando *sim()*, y extraer los datos del *Workspace*.
- **Evaluación de Métricas:** Aplique la función *stepinfo()* sobre los datos simulados para extraer y mostrar en la ventana de comandos el tiempo de asentamiento real y el sobreimpulso exacto.
- **Dashboard de Gráficas:** Despliegue en una sola figura bien organizada:
 - a. La respuesta temporal del nivel del tanque vs. la referencia y la señal del esfuerzo de control.
 - b. El gráfico del Lugar Geométrico de las Raíces del lazo corregido.

- c. El Diagrama de Bode con los márgenes de estabilidad (GM y PM) explícitamente anotados por el software.

Entregables

1. **Código Fuente (.m):** Script de MATLAB completamente comentado que resuelva la linealización simbólica, sintonización, simulación y postprocesamiento.
2. **Modelo de Simulink (.slx):** Diagrama de bloques dinámico con el controlador PID configurado con anti-windup y las perturbaciones especificadas.
3. **Informe Ejecutivo:** Capturas del dashboard gráfico generado y un análisis crítico de ingeniería que confirme si los márgenes de estabilidad (GM y PM) son lo suficientemente seguros para autorizar la puesta en marcha del lazo de control en una facilidad petrolera real.

Desarrollo:

Iniciamos linearizando el modelo alrededor del punto de operación.

```
%Usamos jacobian()
syms A Qi Cv h h0
%Despejando de la EDO:
f_eq_modelo = (Qi- Cv*(sqrt(h)))/A;
% Calculamos las matrices jacobianas
%sea a la derivada parcial de f_eq_modelo respecto a h
%sea b la derivada parcial de f_e1_modelo respecto a Qi
a_sym = jacobian(f_eq_modelo, h)
```

$$a_{sym} = -\frac{C_v}{2 A \sqrt{h}}$$

```
b_sym = jacobian(f_eq_modelo, Qi)
```

$$b_{sym} = \frac{1}{A}$$

De esta forma obtenemos las derivadas parciales. Ahora se evalúa sobre el punto de evaluación.

```
a_operativo = subs(a_sym, h, h0)
```

$$a_{operativo} = -\frac{C_v}{2 A \sqrt{h_0}}$$

```
b_operativo = subs(b_sym, h, h0)
```

$$b_{operativo} = \frac{1}{A}$$

```
%Asumiendo un rango industrial de 3.14 m2 a 12.56 m2 de área
%transversal:
A_tanque = 4.91;
%Con un rango industrial típico de 50 a 300 gpm
Cv_tanque = 7.57*10^-3;
h0_operativo = 2; %Nos lo da el enunciado
a_operativo_eq = subs(a_operativo, [h0, A, Cv], [h0_operativo, A_tanque,
Cv_tanque])
```

```
a_operativo_eq =
```

$$-\frac{757\sqrt{2}}{1964000}$$

```
b_operativo_eq = subs(b_operativo, [h0, A, Cv], [h0_operativo, A_tanque,
Cv_tanque])
```

```
b_operativo_eq =
```

$$\frac{100}{491}$$

```
a_operativo_eq = double(a_operativo_eq);
b_operativo_eq = double(b_operativo_eq);
tf_proyecto = tf(b_operativo_eq, [1 -a_operativo_eq])
```

```
tf_proyecto =
```

$$\frac{0.2037}{s + 0.0005451}$$

```
Continuous-time transfer function.
Model Properties
```

Con esto podemos dar paso a la siguiente fase del proyecto:

En casos anteriores hemos encontrado las ganancias para los controladores de forma analítica, pero ahora usaremos las herramientas nativas de Matlab.

```
[C_pid, info_C_pid] = pidtune(tf_proyecto, "PID")
```

```
C_pid =
```

$$K_p + K_i * \frac{1}{s}$$

```
with Kp = 0.00383, Ki = 3.79e-06
```

```
Continuous-time PI controller in parallel form.
```

```
Model Properties
```

```
info_C_pid =
```

```
struct with fields:
```

```
Stable: 1
CrossoverFrequency: 9.7082e-04
PhaseMargin: 73.8101
```

Notamos un Frecuencia de cruce muy baja, lo que podría traducirse en una respuesta más lenta de lo que necesitamos (30s)

```
t_proyecto_sim = 0:01:300;  
sys_proyecto = feedback(C_pid*tf_proyecto, 1)
```

```
sys_proyecto =
```

$$\frac{0.0007803 \text{ s} + 7.71\text{e-}07}{s^2 + 0.001325 \text{ s} + 7.71\text{e-}07}$$

Continuous-time transfer function.
Model Properties

```
info_sys_proyecto = stepinfo(sys_proyecto)
```

```
info_sys_proyecto =  
struct with fields:  
    RiseTime: 1.6734e+03  
    TransientTime: 5.8071e+03  
    SettlingTime: 5.8071e+03  
    SettlingMin: 0.9005  
    SettlingMax: 1.0608  
    Overshoot: 6.0819  
    Undershoot: 0  
    Peak: 1.0608  
    PeakTime: 3.6135e+03
```

```
[y_proyecto_tuned, t_proyecto_tuned] = step(sys_proyecto);  
figure;  
plot(t_proyecto_tuned, y_proyecto_tuned, "LineWidth", 2, "Color", [0 1 0.4])  
title("Respuesta al paso del tanque")  
xlabel("Segundos (s)")  
ylabel("Altura del tanque (m)")
```

Esto nos dice que no cumplimos con la meta, ya que se demora más de 5800 segundos. Lo que equivale a más de una hora y media. El comando pidtune() permite alterar las ganancias en función de un ancho de banda.

```
%BW = 4/Ts (aprox) (wc)  
ts_proyecto = 600;  
OS_proyecto = 0;  
wc = 4/ts_proyecto;  
[C_pid, info_C_pid] = pidtune(tf_proyecto, "PID", wc)
```

```
C_pid =
```

$$K_p + K_i * \frac{1}{s}$$

with $K_p = 0.0316$, $K_i = 5.96\text{e-}05$

Continuous-time PI controller in parallel form.
Model Properties
info_C_pid =

```

struct with fields:
    Stable: 1
    CrossoverFrequency: 0.0067
    PhaseMargin: 78.8721

```

```
sys_proyecto = feedback(C_pid*tf_proyecto, 1)
```

```

sys_proyecto =

    0.006436 s + 1.214e-05
    -----
    s^2 + 0.006981 s + 1.214e-05

```

Continuous-time transfer function.
Model Properties

```
info_sys_proyecto = stepinfo(sys_proyecto)
```

```

info_sys_proyecto =
struct with fields:
    RiseTime: 240.5550
    TransientTime: 1.4660e+03
    SettlingTime: 1.4660e+03
    SettlingMin: 0.9142
    SettlingMax: 1.0946
    Overshoot: 9.4553
    Undershoot: 0
    Peak: 1.0946
    PeakTime: 630.2419

```

```

[y_proyecto_tuned, t_proyecto_tuned] = step(sys_proyecto);
figure;
plot(t_proyecto_tuned, y_proyecto_tuned, "LineWidth", 2, "Color", [1 0 0.4])
title("Respuesta al paso del tanque")
xlabel("Segundos (s)")
ylabel("Altura del tanque (m)")

```

Aunque se observa una notable mejora ($t_s = 93s$), es necesario hacer el controlador de forma analítica ya que nos encontramos lejos del t_s objetivo y hay un overshoot mayor a 0%.

```

%Encontramos los polos deseados
%ts = 4/zeta*wn ; wn = 4/zeta*ts; zeta = 1
%para zeta = 1/ s1,s2 = -wn
wn_proyecto = wc

```

```

wn_proyecto =
0.0067

```

```
parte_real_proyecto = -wn_proyecto
```

```

parte_real_proyecto =
-0.0067

```

```

figure;
rlocus(sys_proyecto)
sgrid(0.99,[]);
%Para hallar las ganancias
% Siendo el denominador en general de una TF:  $1 + C(s) \cdot G(s) = 0$ 

```

```

%C(s) = (Kps +Ki)/s
%G(s) = b/(s-a)
%Resolviendo con symbolab:
%(s^2 - as) \cdot 1 + (bK_p s + bK_i) = 0
% s^2 + (b K_p - a)s + b K_i = 0
%Para la ecuación característica basadas en nuestras raíces:
%s=r, entonces el factor es (s-r)
%Como hay dos raíces s1 y s2 = wn_proyecto = 0.1333
%s^2 + 2(0.1333)+ (0.1333)^2: esta es la deseada

% (b K_p - a) = 2(wn_proyecto);
% b K_i = (wn_proyecto)^2;
k_p_proyecto = (2*wn_proyecto + a_operativo_eq ) / b_operativo_eq

```

```

k_p_proyecto =
0.0628

```

```

k_i_proyecto = (wn_proyecto^2)/b_operativo_eq

```

```

k_i_proyecto =
2.1822e-04

```

```

C_pid = pid(k_p_proyecto, k_i_proyecto, 0)

```

```

C_pid =

```

$$K_p + K_i * \frac{1}{s}$$

```

with Kp = 0.0628, Ki = 0.000218

```

```

Continuous-time PI controller in parallel form.
Model Properties

```

```

sys_proyecto = feedback(C_pid*tf_proyecto, 1)

```

```

sys_proyecto =

```

$$\frac{0.01279 s + 4.444e-05}{s^2 + 0.01333 s + 4.444e-05}$$

```

Continuous-time transfer function.
Model Properties

```

```

info_sys_proyecto = stepinfo(sys_proyecto)

```

```

info_sys_proyecto =
struct with fields:
    RiseTime: 117.4845
    TransientTime: 787.9945
    SettlingTime: 787.9945
    SettlingMin: 0.9181
    SettlingMax: 1.1137
    Overshoot: 11.3667
    Undershoot: 0
    Peak: 1.1137
    PeakTime: 310.8490

```

```
[y_proyecto_tuned, t_proyecto_tuned] = step(sys_proyecto);
figure;
plot(t_proyecto_tuned, y_proyecto_tuned, "LineWidth", 2, "Color", [0.4 0 1])
title("Respuesta con Control PI (Analítico)")
xlabel("Segundos (s)")

ylabel("Altura del tanque (m)")
```

Dado que las anteriores estrategias no funcionaron, se realizará un esquema I-P. Este esquema hace que la acción proporcional mire directamente hacia la medición del tanque y no al error. La acción integral entra a trabajar entonces de forma lenta y progresiva, que es lo que se necesita en este tipo de aplicaciones.

Esto mejora la vida útil de los actuadores, al evitar acciones excesivas de la ganancia proporcional, los movimientos bruscos ya no destruirán sellos y se disminuye desgaste en posicionadores, elementos móviles, etc.

```
ts_proyecto = 600;
OS_proyecto = 0;
wc = 4/ts_proyecto;
wn_proyecto = wc
```

```
wn_proyecto =
0.0067
```

```
k_p_proyecto = (2*wn_proyecto + a_operativo_eq ) / b_operativo_eq
```

```
k_p_proyecto =
0.0628
```

```
k_i_proyecto = (wn_proyecto^2)/b_operativo_eq
```

```
k_i_proyecto =
2.1822e-04
```

```
sys_proyect_interno = feedback(tf_proyecto, k_p_proyecto);
num_C_integral = k_i_proyecto;
den_C_integral = [1 0];
C_integral = tf(num_C_integral, den_C_integral)
```

```
C_integral =
```

```
0.0002182
-----
s
```

Continuous-time transfer function.
Model Properties

```
sys_IP_esquem = feedback(C_integral * sys_proyect_interno, 1)
```

```
sys_IP_esquem =
```

```
4.444e-05
-----
s^2 + 0.01333 s + 4.444e-05
```


Continuous-time transfer function.
Model Properties

```
info_IP_esquem = stepinfo(sys_IP_esquem)
```

```
info_IP_esquem =  
struct with fields:  
    RiseTime: 503.7506  
    TransientTime: 875.1168  
    SettlingTime: 875.1168  
    SettlingMin: 0.9019  
    SettlingMax: 0.9999  
    Overshoot: 0  
    Undershoot: 0  
    Peak: 0.9999  
    PeakTime: 1.7891e+03
```

```
[y_IP, t_IP] = step(sys_IP_esquem);  
figure;  
plot(t_IP, y_IP, "LineWidth", 3, "Color", [0 1 0.3]);  
hold on;  
yline(1, '--', 'Color', [0.7 0.7 0.7], 'LineWidth', 1.5); % Línea punteada  
gris del Setpoint  
title("IP Esquema Response (Interface Level Control)")  
xlabel("Time (s)")  
ylabel("Interface Level (m)") % Vocabulario técnico de Oil & Gas  
grid on;
```

Con esto se queda dentro del rango de entre 13 y 15 minutos que son estándar en la industria con un overshoot de 0%.

La implementación de un controlador PI introdujo un cero en la función de transferencia, generando un overshoot de 11.36%. Fue por ello que se optó por implementar un esquema I-P, la cual elimina el cero del controlador y contribuye a mitigar el sobre impulso, lo cual es indispensable en aplicaciones de este tipo.

La selección de este esquema mitiga el desgaste mecánico dado que la ganancia proporcional actúa sobre la variable de proceso y no sobre el error, lo que se traduce en el ahorro en costos de mantenimiento.

Otra a cosa a destacar, fue la discrepancia entre el target Settling Time (600 s) y reportado por el esquema I-P (875.11 s). esto se hizo utilizando la ecuación $t_s = \frac{4}{\omega_n}$, la cual asume un sistema de segundo orden

estándar. No obstante, el esquema IP suprime el cero del numerador, es decir, se eliminó la componente con la que contaba t_s para alcanzar la respuesta transitoria. Esto es preferible en este tipo aplicaciones ya que respuestas aceleradas tendrían consecuencias como agua saliendo por la línea del crudo (water carryover) o crudo yéndose con el agua de desecho (oil carryunder), lo cual viola normativas ambientales.

Para analizar la robustez del sistema, se utilizan los dos métodos planteados, para lo cual tenemos que abrir primero el sistema

```
open_loop_sys = C_integral * sys_proyect_interno
```

```
open_loop_sys =
    4.444e-05
    -----
    s^2 + 0.01333 s
```

Continuous-time transfer function.
Model Properties

```
figure;
rlocus(open_loop_sys)
sgrid(0.99, [])
figure;
options_proyect = bodeoptions;
options_proyect.Grid = "on";
%margin(open_loop_sys, options_proyect)
[Gm, Pm, Wcg, Wcp] = margin(open_loop_sys)
```

```
Gm =
Inf
Pm =
76.3454
Wcg =
NaN
Wcp =
0.0032
```

Tenemos un margen infinito ($G_m = \infty$), así como una frecuencia de cruce de ganancia $\omega_{cg} = \text{NaN}$, certifica que el sistema no caerá en inestabilidad debido a aumentos. Teniendo en cuenta como estándar un Phase Margin de 45° , este sistema lo supera ampliamente con un margen de fase de 76.36° . Esto asegura que el sistema funcione de forma correcta ante altos niveles de retardo.